

**Sri Lanka Institute of
Information Technology**

Secure Operating Systems - IE2032



**Developing Process Scheduling Policies in
User Space With Common OS Features**

Process Scheduling in Operating Systems

2025.04.28

Group Details

Group Number – SOS_35

Name	Student ID
DENCIL P A N	IT23545212
NEHANYA M D V	IT23763708
DISSANAYAKE D M H W	IT23711082
SANJANA V A H	IT23560680

Table of Contents

Terms Of Reference	3
Abstract.....	4
Introduction and Significance.....	5
Methodology	6
Concepts Of Modern Operating Systems	7
Application Of Ideas to Current or Future Operating Systems.....	7
Evaluation Of Process Scheduling.....	9
Problem Statement and Its Importance.....	10
Proposed Solution	11
Key Components:.....	12
Result and Implications.....	13
Critical Evaluation.....	14
Relate To Border Concepts	15
Conclusion	16
References	17
Individual Contribution	18

Terms Of Reference

A report submitted to Sri Lanka Institute of Information Technology for partial fulfillment of the requirements for the IE2032 module, as part of the degree of Bachelor of Science (Hon's) in Information Technology, specializing in Cyber Security.

Abstract

The paper “*Developing Process Scheduling Policies in User Space with eBPF*” addresses a fundamental challenge in operating system design: kernel-based process scheduling policy inflexibility. Traditional operating systems incorporate scheduling logic directly within the kernel, which complicates developers from experimenting with, testing, or modifying scheduling policies without touching the kernel source code. This rigidity can hinder innovation, particularly in dynamic environments (volatile settings) like cloud-native workloads or real-time systems. To overcome this limitation, the authors introduce a new system that utilizes the Extended Berkeley Packet Filter (eBPF) technology to make it easier to develop and deploy custom scheduling policies in user space. By placing eBPF hooks within the scheduler of the Linux kernel, the platform offers a mechanism for dynamic scheduling logic generated by the use

Introduction and Significance

The scheduling process is an element of multiprogramming is a single CPU core to run many processes at once. OS kernels usually include scheduling policies, and their algorithms. By obtaining sufficient efficiency for a variety of applications. Process scheduling policies designed for certain workloads can enhance application performance.

Issue and its related work:

The problem is processing scheduling policies are difficult to create and implement.

extensions for kernels and hypervisors:

Non-up streaming kernel/hypervisor additions are frequently included in proposals for scheduling enhancement, which creates issues for users regarding security, stability, and future maintenance.

User-space execution times:

It can be difficult to schedule optimizations based on user-space threads, also referred to as coroutines or fibers, to enhance the performance of current programs because of their unique runtimes.

Frameworks for development:

The frameworks that suggest for the establishment of process scheduling policies provide flexibility but necessitate non-up streamed kernel-space additions, which makes system deployment challenging for many users.

Implementing standard OS functions:

The use of preemptive scheduling to reduce tail latency is limited by report that proposes process scheduling methods utilizing normal OS characteristics but lacks adaptability for serverless computing and stream processing.

Customized process scheduling policies for workloads to show the advantages, nevertheless their implementation is difficult because there isn't a straightforward method in place. The priority elevation trick in microsecond-scale time slicing, table-driven scheduling, and preemptive scheduling are some of the programming techniques used in this study to design process scheduling policies in user space utilizing conventional OS capabilities.

Significance

This is addressed by the paper, Developing Process Scheduling Policies in User Space with Common OS Features, which proposes a way in which scheduling policies may be created in user space without altering the kernel. This milestone is important since it enables easy, secure, and pervasive customizing of scheduling behavior, making high-end performance tuning accessible to developers who are not familiar with kernel-level programming.

Methodology

The authors designed and implemented a system by which scheduling choices may be guided or overridden by runtime-loaded eBPF programs.

Key components include:

- **User-space software** for defining, compiling, and loading eBPF policies.
- **Kernel-side hooks** are inserted at places in the scheduler where decisions are made.
- **A verification engine** guarantees that eBPF programs uploaded to the kernel are safe and terminated correctly.

The system was validated by experimenting on real systems. Metrics such as context-switching overhead, scheduling latency, and throughput were compared and measured across multiple workloads with both traditional and eBPF-based scheduling methods.

Concepts Of Modern Operating Systems

Even now, current operating systems aim for flexibility and reliability. But a lot of modern applications (such as cloud services, IoT, and high-frequency trading systems) were designed with very particular performance requirements. This paper demonstrates that **developers can:**

- Tailor the scheduling policy without modifying the kernel.
- Control processes to a granularity of microseconds, as required by most current systems.
- Optimizations are implemented for both native and virtual machines.

So, the paper's concepts align nicely with where the future of OS development is going.

Application Of Ideas to Current or Future Operating Systems



The concepts in this paper can be applied in various ways:

1. Practical scheduling without kernel modifications

Developers can implement user-space schedulers that control the process scheduling order, which can be optimized for better performance, without requiring kernel patches.

2. Improved performance for dedicated servers

This can be used to reduce latency and handle high loads better, for applications like web servers, databases or real-time data processing.

3. Fine-grained Resource Control

In contexts like cloud or IoT, small, controlled slices of CPU time can help to accomplish many tasks well on almost the same core.

4. Whence the Real Time / High Frequency Systems Support

Systems requiring absorption-tight timing (e.g., stock trading systems) can take advantage of scheduling at microsecond scale.

5. User Experimentation for New Scheduling Policies in the User-Space

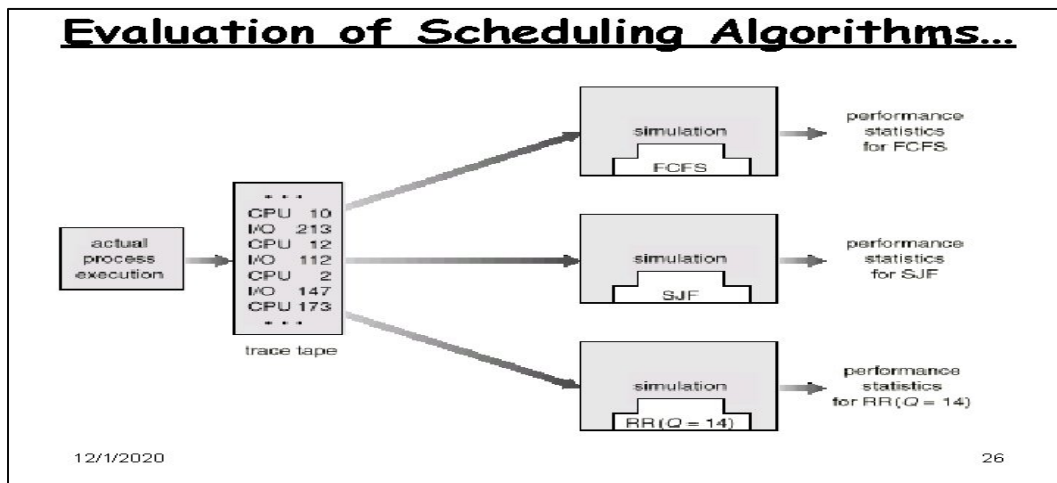
Developers and researchers can easily experiment with new scheduling ideas in the user space, without risk of crashing the OS.

Such user-space control might get even richer official support by operating systems in the future, marrying the safety of a more disciplined language with flexibility to a greater degree.

Evaluation Of Process Scheduling

Intel Xeon Gold 6326 CPUs and 128 GB DRAM are available for the two networking explore computers. For VM analyze, Mellanox ConnectX-5 100 Gbps NICs are used. For both the virtual and bare metal computers, Linux 6.2 is configured. In bare-metal computers, the kernel-space process scheduler control SCHED_FIFO-applied scheduling entities when execution time exceeds a approach. This report covered the under microsecond CSA time slicing, table driven scheduling, preemptive process scheduling, overhead of process scheduling.

This report investigates the upward priority raising trick in scheduling. It compares the delays of two standard scheduling entities: two processes and two virtual CPUs on a single physical CPU core. In the assign pattern case, the waiting time is until five microseconds have passed, while in the shared pattern case, usleep is manipulated to five microseconds. The results suggest that while often specialized patterns incur delays in shared patterns and vice versa, there are unique delays specialized patterns introduce. Two standard scheduling entities: one executing a busy loop on the same CPU core, and the other the Sysbench CPU benchmark, are performed in parallel to assess the CPU elevation loop of the priority raising trick. While the concerned pattern contains lesser score fall debits, shorter scheduling intervals lead to greater score loss. Dedicated pattern rather than shared pattern results



Problem Statement and Its Importance

Process scheduling is a central operating system (OS) function that governs which processes may utilize the CPU, and in what order. In ancient operating systems, such as Linux, this scheduler is hidden far down in the kernel. Though granting robust control over system behavior and performance, it is problematic for developers and researchers wishing to experiment with or deploy alternative scheduling algorithms.

It is hard, time-consuming, and generally not practical to change the kernel to accommodate new scheduling policies. It requires close familiarity with kernel internals, access to source code, and recompilation and reboot of the system for each change. The high barrier to entry discourages experimentation and inhibits the use of scheduling policies tailored to modern workloads, such as real-time applications, data-intensive processing, or containerized environments.

Furthermore, the single-size-fits-all nature of general-purpose kernel schedulers means that they are often unable to satisfy the needs of particular use cases. Clearly, there is a need for a more flexible, accessible, and safe method of developing and testing new scheduling policies without compromising the stability of the operating system.

Proposed Solution

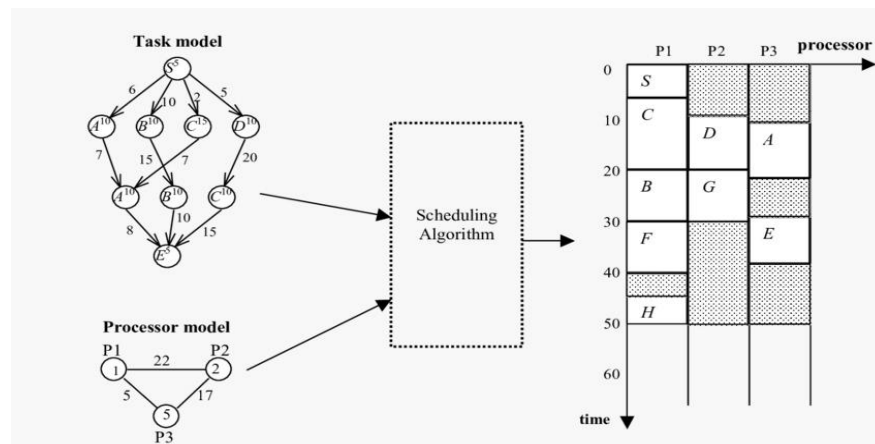
To overcome these limitations, the authors introduce a new framework that allows developers to execute and test process scheduling policies in user space—beyond the kernel—using available operating system functionality and the eBPF (extended Berkeley Packet Filter) technology.

The basic idea is to separate the scheduling decision logic (which is in user space) from enforcing those decisions (which remain in the kernel). The framework uses system calls and capabilities like cgroups, nice levels, and `sched_setattr()` to influence kernel scheduling behavior based on user-space scheduler directives.

eBPF is used for monitoring system events and collecting runtime details of running processes in real-time. User-space scheduling selects this data from it and subsequently adjusts priorities as well as resource delegation based on user-defined policies in a dynamic manner. Because policy implementations do not exist within the kernel, policies become easy to update or replace in no-down, non-stop time.

Utilizing this approach enables us to achieve three extremely beneficial goals:

1. **Flexibility** – Schedulers can be configured and deployed with several scheduling policies for different workloads.
2. **Safety** – Since the kernel is not changed, system stability is ensured.
3. **Ease of Use** – The framework removes the complication of testing novel scheduling approaches, simplifying OS research.



Key Components:

1. User-Space Scheduling Logic:

- Developers can implement custom scheduling algorithms in user space, allowing for rapid prototyping and experimentation.
- The scheduling logic can be tailored to specific application needs, such as prioritizing latency-sensitive tasks or balancing CPU and I/O-bound processes.

2. Kernel-Level Enforcement:

- While the scheduling decisions are made in user space, their enforcement is carried out by the kernel using existing mechanisms like cgroups, nice values, and `sched_setattr()`.
- This ensures that the custom scheduling policies are applied consistently and efficiently without compromising system stability.

3. eBPF Integration:

- eBPF is utilized to monitor system events and collect performance data about running processes in real time.
- This data is passed to the user-space scheduler, which can dynamically adjust priorities and resource allocation according to custom policies.
- eBPF's ability to safely execute code within the kernel space allows for efficient data collection without introducing significant overhead.

Result and Implications

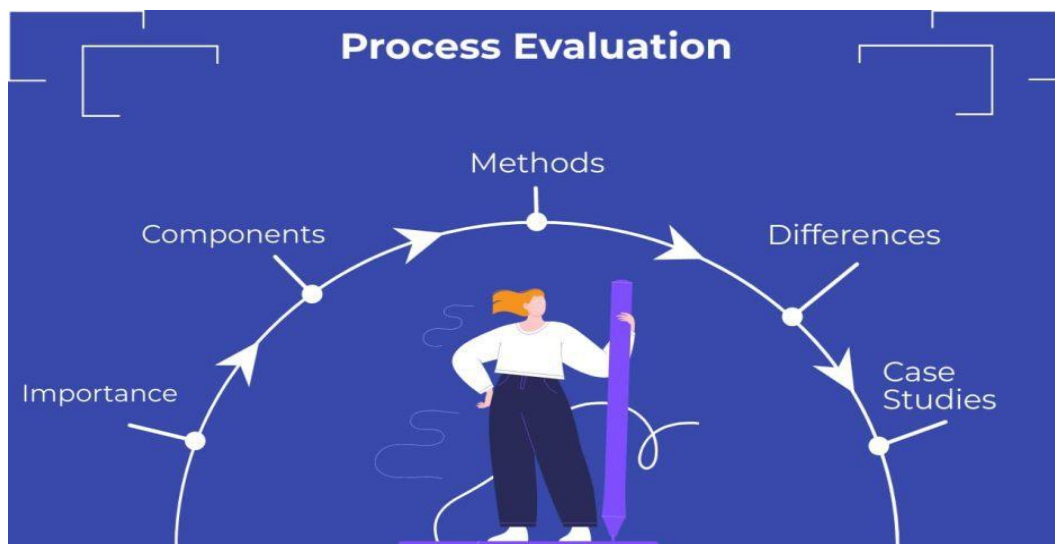
The experiment proved that this approach has tangible benefits:

- Flexibility: New scheduling policies were written, installed, and replaced at runtime.
- Security and Stability: eBPF's verifier prevented malicious code from being loaded.
- Performance: Low overhead was introduced; the impact on scheduling latency was negligible.
- Usability: The authors provided a tool chain for writing and installing eBPF policies, significantly lowering the barrier to kernel-level experimentation.

The consequences are considerable: operating systems could shift away from a hard, static state to a flexible, modular state where kernel actions are adaptive. It enables experimenting in production and more responsiveness to changing application demands.

Critical Evaluation

- **Strengths:**
 - Highly innovative and practical.
 - Maintains strong safety guarantees via eBPF.
 - Brings theoretical OS design principles into real-world practice.
- **Weaknesses:**
 - eBPF's instruction set and verifier limitations restrict policy complexity.
 - Portability to non-Linux OS is limited.
 - Debugging eBPF logic can be difficult due to limited tooling.
- **Future Research Areas:**
 - Expanding eBPF capabilities to support more complex scheduling behaviors.
 - Integration with machine learning for adaptive, real-time policy optimization.
 - Developing better tools for writing, testing, and validating eBPF policies.



Relate To Border Concepts

Contribution to OS Understanding:

This paper enhances the understanding of process scheduling by showing how policy decisions can be abstracted and manipulated safely from user space. It challenges the traditional kernel-bound approach and illustrates how modern tools can extend operating systems' modularity.

Relevance to Modern OS:

Modern systems, especially cloud-native environments, require dynamic adaptation to workload changes. The presented approach is directly relevant, as it enables per-container or per-application scheduling customization. It aligns with the growing trend toward programmable infrastructure.

Applications in Current/Future Systems:

- **Cloud Providers:** Could use this for tenant-specific scheduling needs.
- **IoT and Real-Time Systems:** Custom schedulers can optimize for latency or energy usage.
- **Research and Education:** Provides a powerful way to teach OS concepts practically.



Conclusion

The paper in "Developing Process Scheduling Policies in User Space with Common OS Features" presents a novel and valuable technique—the priority elevation trick—to develop and deploy user space bespoke scheduling policy implementation without kernel-level alterations to the OS. Kernel-level alteration of scheduling historically was hard, high-risk, and unrealistic for most programmers, slowing innovation, especially in deploying OS behavior to specialized workloads. This paper demonstrates that with careful manipulation of process priority using typical system calls like `sched_setscheduler` and `sched_setaffinity`, it is possible to build flexible, secure, and highly interactive scheduling systems from purely user space.

By extensive experiments, the authors showed the feasibility of microsecond-scale time slicing, table-driven scheduling, and preemptive scheduling and established significant performance gains for modern applications like networked servers. This work is highly applicable to modern and future operating systems, especially in areas like cloud computing, real-time systems, and IoT, where workload-specific and dynamic optimization is crucial.

By enabling low-overhead, high-flexibility scheduling without sacrificing system stability, the work significantly expands the practical boundaries of operating system design. Furthermore, it promotes safer experimentation, faster innovation cycles, and is in line with trends toward more programmable, responsive infrastructures.

References

- **Research paper** - Developing Process Scheduling Policies in User Space with CommonOSFeatures - <https://dl.acm.org/doi/epdf/10.1145/3678015.3680481>
- Kenichi Yasukata and Kenta Ishiguro, *Developing Process Scheduling Policies in User Space with Common OS Features*, 15th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys '24), 2024. DOI: [10.1145/3678015.3680481](https://doi.org/10.1145/3678015.3680481).
- Linux Programmer's Manual, *sched_setscheduler(2)* and *sched_setaffinity(2)* system call documentation, available at man7.org.
- The Linux Foundation, *Understanding Linux Scheduling Policies*, available at kernel.org documentation.
- Extended Berkeley Packet Filter (eBPF) official documentation, ebpf.io.

Individual Contribution

Student ID	Student Name	Individual Contribution
IT23545212	Dencil P.A. N	1. Problems and Solutions 2. Results and Implications 3. Conclusion
IT 23 763708	Nehanya M.D. V	1. Abstract 2. Methodology 3. Critical Evaluation 4. Conclusion
IT 23711082	Dissanayake D.M.H.W	1. Concepts and application 2. Border Concepts
IT23560680	Sanjana V.A.H	1. Introduction and Significance 2. Evaluation